

Agentic RAG Chatbot

Developer guide

A practical guide to developing, running and extending the local frontend documentation assistant.

Python 3.12 / LangGraph / Streamlit / Ollama / Chroma

Proof of concept · version 0.1.0

Covers the RAG reliability update and its measurements of 4 October 2026; the measurements of 3 October 2026 serve as the baseline.

4 October 2026

Contents

01	Project scope and design	3
02	Repository tour	4
03	Local development setup	5
04	Fake mode and command-line use	6
05	Docker Compose operation	7
06	Runtime architecture	8
07	Routing, planning and verification	9
08	State contracts and Python API	10
09	Corpus download and preprocessing	11
10	Index lifecycle and migration	12
11	Hybrid RAG retrieval	13
12	Deterministic tool contracts	14
13	Configuration: models and storage	15
14	Configuration: workflow controls	16
15	UI, streaming and trace events	17
16	Testing and continuous integration	18
17	Extending the application	19
18	Functional evaluation	20
19	Performance and measured results	21
20	Troubleshooting and runbook	22
21	Maintenance and source reference	23

Commands run from the repository root. Shell-specific examples are labelled. Code identifiers, environment variables and file paths are identical in both language editions.

01 Project scope and design

The Agentic RAG Chatbot is a Python proof of concept for a frontend developer assistant. It answers from downloaded, pinned snapshots of official documentation and uses local, deterministic tools for facts that can be calculated or looked up exactly.

The knowledge base covers MDN Web Docs, React, Vue, Next.js, Nuxt and the TypeScript Handbook. The user can ask in English or Hungarian. Retrieval rewrites a question into English; normal answer synthesis follows the language identified by the router. Exact tool output and some fallback messages retain their implemented English wording.

What the system demonstrates

- Seven main LangGraph nodes, conditional routing and parallel, independently executed sub-tasks.
- A separate four-node RAG subgraph, called through the knowledge-base search tool.
- Local inference through Ollama, local CPU embeddings and a persistent Chroma index.
- A Streamlit chat interface showing workflow steps and retrieved evidence.
- Repeatable ingestion, offline tests, functional evaluation and a load-test harness.

Important implementation boundaries

The application is a prototype, with a chat UI and CLI rather than a separate public REST API. Graph state is fresh for each request: there is no LangGraph checkpoint or durable conversation store. The UI supplies completed conversation history to subsequent calls.

The fake providers are development fixtures. They verify execution paths without a model server; their answers and retrieval scores do not establish model quality. Initial corpus downloads still need network access, even with fake providers.

The repository uses the MIT license for its code. The source manifest records separate licenses for downloaded documentation. The corpus is downloaded rather than committed.

Basis: `README.md`, `pyproject.toml`, `agent/graph.py`, `agent/state.py`, `data/sources.toml`.

02 Repository tour

Application code lives under `src/agentik_rag/`. Import the installed package; avoid changing `sys.path` in application code. The console entry point is `agentik-rag = agentik_rag.cli:main`.

Location	Responsibility
<code>config.py, errors.py</code>	Validated settings and shared application exceptions.
<code>cli.py, __main__.py</code>	Commands, argument validation, logging and process exit codes.
<code>llm.py, embeddings.py</code>	Real and fake providers; local model adapters.
<code>agent/graph.py, routing.py</code>	Main graph wiring and pure conditional routing.
<code>agent/nodes.py, prompts.py, state.py</code>	Node behavior, structured prompts and state contracts.
<code>agent/tools.py</code>	Search tool and three deterministic tool wrappers.
<code>agent/contrast.py, specificity.py, compat.py</code>	Domain calculations and compatibility-data lookup.
<code>rag/</code>	Query rewrite, retrieval, grading, cited context and BM25 fusion.
<code>ingestion/</code>	Source manifest, download, cleaning, chunking, indexing and startup preparation.
<code>ui/</code>	Streamlit entry point and presentation components.
<code>evaluation/, loadtest/, reports.py, tracing.py</code>	Datasets, metrics, reports and shared instrumentation.
<code>tests/, .github/workflows/ci.yml</code>	Offline verification and CI.

Data and infrastructure

`data/sources.toml` and the evaluation JSONL files are versioned inputs. Downloaded sources in `data/raw/` and vectors in `data/chroma_db/` are generated, gitignored data. `data/eval/results/` contains saved measurement reports.

`pyproject.toml`, `uv.lock` and `.python-version` define the Python environment. `Dockerfile`, `compose.yaml` and the optional `compose.gpu.yaml` define container operation. `.streamlit/config.toml` configures the UI theme.

Paths in later chapters are relative to `src/agentik_rag/` unless they begin with `data/`, `docs/`, `tests/` or a repository-level filename.

03 Local development setup

Use Python 3.12: `pyproject.toml` requires `>=3.12,<3.13`. Install Git and uv; use Ollama for real answers. The lock file selects CPU-only PyTorch because GPU resources are reserved for the chat model.

Set up the checkout

The following commands work in PowerShell and in a POSIX shell. Run them from the repository root after cloning.

```
git clone https://github.com/Csabikahhh/agentic-rag-chatbot-poc.git
cd agentic-rag-chatbot-poc
uv sync --locked
uv run agentic-rag --help
uv run agentic-rag config
```

`uv sync --locked` installs the project and development tools without silently updating an inconsistent lock file. For Windows configuration, use `Copy-Item .env.example .env`; on Linux or macOS use `cp .env.example .env`.

Run the real local stack

Start the Ollama desktop application or run `ollama serve` in a separate terminal. Then use:

```
ollama pull qwen3.5:4b
uv run agentic-rag ingest --download
uv run agentic-rag serve
```

Open `http://localhost:8501`. The default Ollama URL on the host is `http://localhost:11434`. First ingestion downloads pinned documentation and the embedding model; first retrieval in a process loads the embedding model.

`serve` performs startup ingestion by default. The explicit `ingest --download` above makes preparation visible and easier to diagnose. Directly running `uv run streamlit run src/agentic_rag/ui/app.py` launches the UI without the CLI's startup preparation; prepare the index first in that case.

Prerequisites and commands: `README.md`; Python bounds and entry point: `pyproject.toml`; startup behavior: `cli.py`.

04 Fake mode and command-line use

Fake mode uses a scripted chat provider and deterministic hashing embeddings. Set both providers together and use a separate index directory when switching locally, so that a fake index does not replace a real embedding index.

PowerShell example

```
$env:LLM_PROVIDER = "fake"
$env:EMBEDDING_PROVIDER = "fake"
$env:CHROMA_DIR = "data/chroma_db_fake"
uv run agentic-rag ingest --download
uv run agentic-rag serve
```

For a POSIX shell, use `export LLM_PROVIDER=fake EMBEDDING_PROVIDER=fake` and `export CHROMA_DIR=data/chroma_db_fake` before the same commands. If the corpus already exists, plain `ingest` avoids downloading it.

PowerShell environment variables persist for the terminal session and take precedence over `.env`. Before returning to the real defaults, remove these overrides with `Remove-Item`

`Env:LLM_PROVIDER, Env:EMBEDDING_PROVIDER, Env:CHROMA_DIR`, or explicitly set the desired real values.

CLI reference

Command	Use
<code>config</code>	Print the effective settings as environment assignments.
<code>ingest</code>	Synchronize the index with the local corpus. <code>--download</code> fetches sources first; <code>--rebuild</code> replaces the collection.
<code>serve</code>	Prepare the knowledge base and launch Streamlit. Supports <code>--address</code> and <code>--port</code> .
<code>eval</code>	Evaluate the full graph or a supported isolated node.
<code>loadtest</code>	Measure graph requests with bounded concurrency.
<code>export-graph</code>	Export the compiled main graph, RAG graph or both as Mermaid.

```
uv run agentic-rag export-graph --output tmp/graphs.md
uv run agentic-rag export-graph --graph rag --format mermaid
```

`--sources PATH` belongs to `ingest --download`. Raw Mermaid output requires a single graph; `--graph all --format mermaid` is invalid. Each command has its own `--help`.

05 Docker Compose operation

The default stack has `app`, `ollama` and a one-shot `ollama-pull` service. The app waits for a healthy server and a completed model pull. The UI is published on loopback port 8501; Ollama is accessible inside the Compose network without a host port.

```
docker compose up --build
docker compose logs -f app
```

The app uses `INGEST_ON_START=true` to download sources and prepare its index before Streamlit starts. On the first run, a long `health: starting` period can reflect legitimate downloads and embedding work. The Dockerfile allows a 20-minute healthcheck start period; this is a healthcheck allowance, not a fixed startup time.

Named volume	Stored data
<code>ollama-data</code>	Downloaded chat-model weights.
<code>corpus-data</code>	Corpus at <code>/app/data/raw</code> .
<code>chroma-data</code>	Indexes under <code>/app/data/chroma_db</code> , separated by embedding provider.
<code>hf-cache</code>	Hugging Face model cache.

Alternative launch modes

```
$env:LLM_PROVIDER = "fake"
$env:EMBEDDING_PROVIDER = "fake"
docker compose up --build --no-deps app
```

For NVIDIA support, use `docker compose -f compose.yaml -f compose.gpu.yaml up --build`. Windows needs Docker Desktop's WSL 2 backend and GPU support. Without the override the Ollama container uses the CPU.

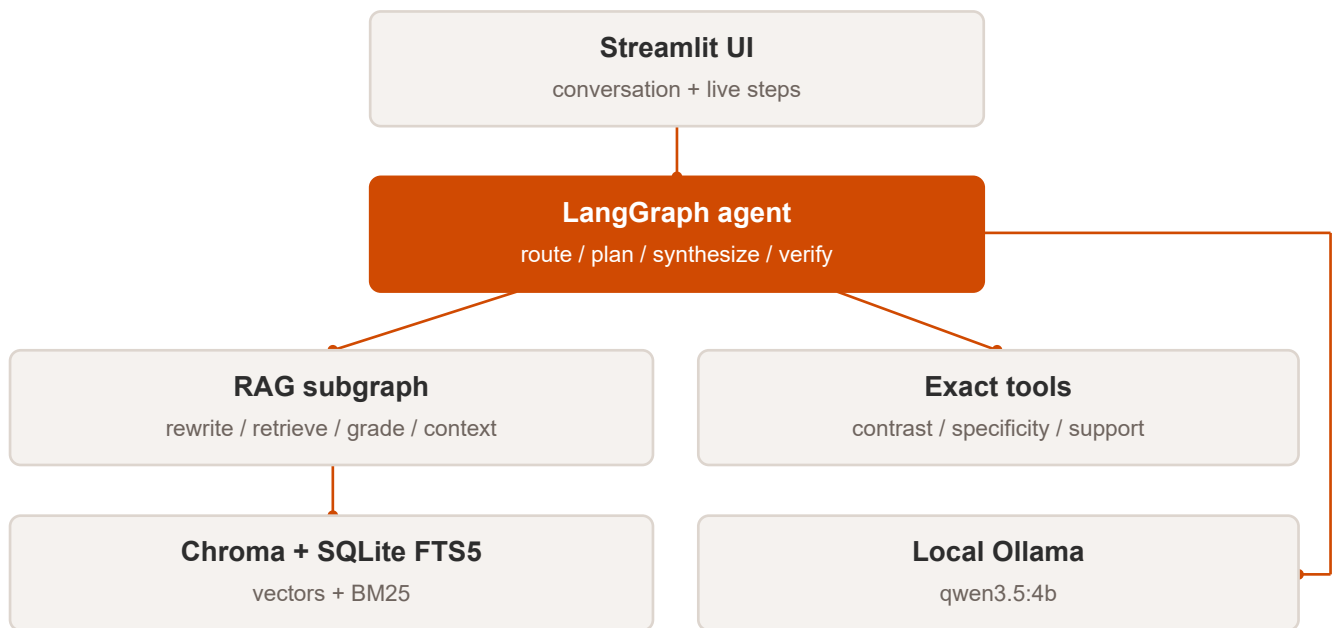
With an existing host Ollama, run:

```
docker compose run --rm --no-deps --service-ports `
-e OLLAMA_BASE_URL=http://host.docker.internal:11434 app
```

The app runs as UID/GID 10001 by default. Linux bind mounts must be writable by that identity; the stack itself uses named volumes. Run evaluations on the host unless you deliberately mount the dataset and result directory.

`docker compose down` keeps volumes. Adding `-v` deletes the corpus, indexes and downloaded models. Source: `compose.yaml`, `compose.gpu.yaml`, `Dockerfile`.

06 Runtime architecture



The Streamlit UI submits conversation messages to the compiled main graph. The agent classifies the request, builds a plan when necessary, invokes retrieval or tools, synthesizes a response and verifies it before finalization.

Dependency boundaries

`build_agent_graph(settings)` binds dependencies once using keyword arguments and `functools.partial`. Nodes receive state as their only positional argument and return partial state updates. Routing functions read state without calling a model or modifying it.

`build_rag_graph(settings)` supplies a lazily opened vector store. A lock protects its first initialization, so parallel searches share one embedding model and index. The BM25 snapshot is also constructed lazily under a lock.

Building either graph does not download a model, connect to Ollama or open an index. This makes graph export and structural tests cheap. Runtime retrieval pays the first-use loading cost.

The nodes are synchronous functions. LangGraph runs parallel `Send` workers in threads; UI streaming, evaluation and the load test use this same synchronous execution path. Converting a node to `async def` requires revisiting the callers rather than changing the function alone.

Basis: `agent/graph.py`, `rag/graph.py`, `rag/lexical.py`. Diagram lines show call/dependency relationships, not a separate network API between Python components.

07 Routing, planning and verification

Intent	Execution path
direct	Router writes a conversational reply; finalization follows immediately.
single	Router creates one retrieval sub-task; dispatch goes directly to the RAG worker.
tool	Router creates one tool sub-task; dispatch goes directly to the tool worker.
complex	Planner decomposes the request into at most five independent sub-tasks.

`plan_subtasks` produces typed tasks. `dispatch_subtasks` emits one `Send` per task to `run_rag_subtask` or `call_tool`. Results fan in before a single `synthesize_answer` execution for the round. A missing intent or missing one-step plan falls back to planning.

Verification outcomes

- **grounded**: finalize the supported draft.
- **insufficient**: re-plan while `retry_count < MAX_RETRIES`; after the limit, finalize with an explicit partial-answer note naming the missing evidence.
- **unavailable**: a structured verification response could not be read. Withhold the draft and ask the user to retry; do not claim that verification succeeded.

`MAX_RETRIES=2` permits two re-plans beyond the initial round. This semantic limit is separate from the transient-error `RetryPolicy(max_attempts=3)` on the four LLM nodes and the RAG worker. Programming errors are not treated as normal transient failures.

Exact single-tool fast path

For one successful built-in tool result, synthesis returns the deterministic output without a model call and routing skips model verification. The allowlist is `check_contrast`, `css_specificity` and `browser_support`. The request still needs the router model call.

The fast path requires a matching task and result, nonempty output, success and no retrieval sources. Failed calls, unknown tools, multiple tools and mixed retrieval/tool requests follow synthesis and verification. Finalization preserves successful tool outputs verbatim and suppresses repeated identical tool outputs across re-plans.

Implementation: `agent/graph.py`, `agent/routing.py`, `agent/nodes.py`, `agent/prompts.py`.

o8 State contracts and Python API

The public main-graph input is `AgentInput`, containing `messages`. Public output includes `answer`, `sources`, `messages`, `question`, `intent`, `subtask_results` and `trace`. Plan and verification fields depend on the route; a direct reply need not have a verifier verdict.

```
from agentic_rag.agent.graph import build_agent_graph
from agentic_rag.config import Settings

settings = Settings(
    _env_file=None,
    llm_provider="fake",
    embedding_provider="fake",
)
graph = build_agent_graph(settings)
result = graph.invoke({"messages": [{"user", "Hello"}]})
print(result["answer"])
```

This greeting example needs no index. A real documentation question requires an index that matches the selected embeddings. Library functions receive settings explicitly; `get_settings()` is reserved for entry points and test fixtures.

Reducers and immutable task models

`messages` uses `add_messages`; `subtask_results` and `trace` use append reducers. Other fields are replaced by writes. A node must return a partial update rather than mutate the input state.

`Subtask` stores `id`, `kind`, `input`, `tool_name` and `tool_args`. A tool task requires a tool name. `SubtaskResult` stores the task identifier, output, ranked sources, success/error and tool name; `error` must be present exactly when `ok` is false. These Pydantic models are immutable values.

Each planning round resets results using `Overwrite(kept)`. Plain append would incorrectly mix rejected-round evidence into the next synthesis. The planner may retain trusted results and omit re-running those tasks.

Citation and conversation contracts

An answer marker `[n]` corresponds to `sources[n - 1]`. RAG workers number locally; synthesis creates global numbering and de-duplicates sources by chunk ID. Finalization removes out-of-range markers.

No checkpointer is configured. Callers pass the relevant conversation history each time. Adding persistence requires resetting per-turn fields and changing callers to send only new messages, otherwise history can be duplicated.

Contracts: `agent/state.py`, `agent/types.py`, `rag/state.py`, `agent/nodes.py`.

09 Corpus download and preprocessing

`data/sources.toml` is the authoritative source list. Each `[[sources]]` entry specifies an ID, display name, repository, full commit SHA, license, root, include/exclude patterns and a public-URL template. A pinned commit makes the indexed corpus reproducible rather than silently following the latest documentation.

```
uv run agentic-rag ingest --download
uv run agentic-rag ingest --download --sources data/sources.toml
```

The downloader uses sparse Git checkouts and copies selected files into `DATA_DIR/<id>/`. A `.source.json` manifest records source metadata. Matching existing manifests allow downloads to be skipped; changing a pinned commit or selection is handled on the next download run.

From source dialects to text

Loaders support Markdown, MDX and plain text. They preserve titles from front matter and normalize MDN macros, JSX, VitePress containers and Nuxt MDC components. Links become readable text and fenced code blocks remain intact during cleaning. Next.js Pages Router blocks are excluded where the current App Router corpus requires it.

Pages are split into H2/H3 sections; the heading path becomes `section` metadata. Dotfiles, dot-directories and README files are skipped. Unsupported file formats fail ingestion rather than being silently ignored.

Structure-aware chunks

- Default body size: 900 characters; overlap: up to 150 characters.
- Pack paragraphs and structural blocks together; carry a heading with the text it introduces.
- Keep fenced code blocks intact up to 1,800 characters; split larger blocks and oversized paragraphs.
- Prefix every chunk with its title and section path, so isolated text still identifies its API and framework.

Chunks preserve `source`, `title`, `section`, `url` and an available manifest `revision`; they also carry chunk identity and offsets. `browser-compat-data` is downloaded with `index=false`: it is input to a tool and never embedded as prose.

Implementation: `ingestion/sources.py`, `download.py`, `markdown.py`, `loaders.py`, `chunking.py`.

10 Index lifecycle and migration

Chroma is a local persistent vector store. The default collection is `documents`; cosine distance is converted to relevance score $1 - \text{distance}$. Embedding provider and model metadata guard the collection against incompatible query vectors.

Routine synchronization

```
uv run agentic-rag ingest
```

Ingestion embeds new or changed chunks and removes stale chunk IDs after the current corpus is indexed. Repeating ingestion over unchanged documents is incremental. An empty corpus fails instead of wiping the existing index.

Changing the embedding model

```
uv run agentic-rag ingest --rebuild
```

Use `--rebuild` after changing `EMBEDDING_PROVIDER` or `EMBEDDING_MODEL` when working with the same collection. Vectors from different models cannot be mixed. Explicit index loading raises `EmbeddingMismatchError` on a mismatch; startup preparation detects that state and rebuilds automatically.

A rebuild replaces the selected collection. Keep separate `CHROMA_DIR` values for experiments when you need to preserve a working baseline. Changing only the chat model does not require a vector-index rebuild.

Reliability-update migration

The current ingestion includes downloaded source commit SHAs in chunk metadata and identity. Run ordinary `ingest` once and restart the app to refresh an older index. The first migration re-embeds affected downloaded chunks because their IDs change. Later unchanged runs remain incremental.

The public documentation URL may display newer content than the indexed snapshot. `Source.revision` identifies the indexed commit; it is not a promise that the public URL serves that revision. Legacy local documents without revision metadata remain usable.

The compiled graph caches its vector-store provider and an in-memory BM25 snapshot. After modifying or re-ingesting the corpus, restart the app or build a new graph. Do not assume an existing lexical snapshot refreshes automatically.

```
Implementation: ingestion/index.py, ingestion/prepare.py, rag/lexical.py; migration:  
docs/rag-improvements.md.
```

11 Hybrid RAG retrieval

The RAG graph always runs `rewrite_query`, `retrieve`, `grade_documents`, then `build_context`. Its public input is `{"query": ...}`; output contains cited context, ranked sources and trace events. The main agent handles re-planning rather than an internal RAG loop.

Rewrite and candidate generation

With Ollama, rewrite produces one short English search query and preserves API/framework names. An empty or overlong rewrite falls back to the original query. Fake mode skips this model call.

The candidate depth is `max(TOP_K, RETRIEVAL_CANDIDATES)`, currently 20. Chroma supplies semantic candidates. With `HYBRID_SEARCH=true`, SQLite FTS5 adds BM25 candidates from the same indexed chunks. Title and section fields receive stronger weighting than body text.

The lexical query includes original and rewritten wording. Tokens are quoted and SQL parameters are bound. This preserves useful identifiers such as `:has`, `$fetch`, underscores and hyphens without interpreting user text as FTS syntax.

Fusion, grading and context

Reciprocal-rank fusion sums $1 / (60 + \text{rank})$ across semantic and lexical lists, then selects the candidate pool. Fusion rank is distinct from cosine similarity. A keyword-only candidate uses an internal score of -1; the exported source similarity is `None` rather than a fabricated number.

The provisional cosine threshold is 0.83 for Hugging Face embeddings and 0.0 for fake embeddings. Lexical matches may bypass this threshold but still reach the configured LLM grader. One structured call grades all candidates that survive thresholding; the final retained set is capped at `TOP_K=4`.

An unreadable relevance-grade response retains threshold/keyword survivors and logs a warning. A model-server connection or timeout error propagates to the parent worker's transient retry policy. This differs from unreadable final-answer verification, which withholds the draft.

`build_context` removes duplicate text and assigns local markers. With no relevant evidence, it returns empty context and sources. Similarity and rank are retrieval signals, not answer-confidence estimates.

Implementation: `rag/graph.py`, `rag/nodes.py`, `rag/lexical.py`.

12 Deterministic tool contracts

Tools are LangChain `BaseTool` instances with Pydantic argument schemas. The planner selects an explicitly named tool and arguments rather than relying on native model tool calls. Unknown fields are rejected by the built-in deterministic input schemas.

Tool	Inputs and behavior
<code>search_knowledge_base</code>	<code>query</code> : invokes the RAG subgraph and returns cited text plus the RAG result artifact.
<code>check_contrast</code>	<code>foreground</code> , <code>background</code> : parses supported CSS colors and computes the WCAG contrast ratio and AA/AAA verdicts. The background must be opaque.
<code>css_specificity</code>	<code>selectors</code> : 1-10 selectors. Computes the (a, b, c) tuple using Selectors Level 4 rules and compares it.
<code>browser_support</code>	<code>feature</code> , optional <code>area</code> and <code>browsers</code> : resolves a web feature in pinned MDN compatibility data and compares supplied browser versions.

Direct invocation example

```
from agentic_rag.agent.tools import get_non_retrieval_tools
from agentic_rag.config import Settings

tools = {tool.name: tool for tool in
          get_non_retrieval_tools(Settings(_env_file=None))}
print(tools["check_contrast"].invoke({
    "foreground": "#777777", "background": "#ffffff"
}))
```

The contrast and specificity calculations need no LLM. Browser support needs downloaded `browser-compat-data` under `DATA_DIR`. That data is loaded lazily; a missing corpus can make the first support call fail even when other tools work.

Interpret results correctly

Contrast verdicts are computed before presentation rounding; the tool displays ratios rounded down. Specificity decides between declarations with the same origin, importance and cascade layer; `!important`, inline styles and layers affect the full cascade. Browser support reflects the pinned compatibility dataset, including relevant support conditions, rather than a live browser test.

Tool validation/domain failures become failed task results. Successful outputs are retained verbatim during finalization, including their language. Mixed retrieval and tool requests still require answer synthesis and verification.

Implementation: `agent/tools.py`, `contrast.py`, `specificity.py`, `compat.py`.

13 Configuration: models and storage

All runtime settings go through immutable `Settings`. Real environment variables override `.env`; empty values mean the default; unknown variables are ignored. Paths resolve from the current working directory. Keep `.env` UTF-8, particularly with Windows PowerShell 5.1.

Variable	Default	Meaning / constraint
LLM_PROVIDER	ollama	ollama or the scripted fake.
OLLAMA_BASE_URL	http://localhost:11434	Server URL on the host; Compose sets <code>http://ollama:11434</code> .
OLLAMA_MODEL	qwen3.5:4b	Local chat-model tag. Pull it before a host run.
OLLAMA_NUM_CTX	8192	Shared prompt/answer context window; 512-131072 tokens.
OLLAMA_TIMEOUT_S	120.0	Positive, finite HTTP timeout in seconds per Ollama request.
OLLAMA_REASONING	false	Thinking option; non-reasoning models ignore it.
LLM_TEMPERATURE	0.0	Sampling temperature, from 0.0 to 2.0.
EMBEDDING_PROVIDER	huggingface	Local sentence-transformers or fake.
EMBEDDING_MODEL	intfloat/multilingual-e5-small	Indexing and querying must use the same model.
DATA_DIR	data/raw	Downloaded and local source corpus.
CHROMA_DIR	data/chroma_db	Persistent vector-index directory.
CHROMA_COLLECTION	documents	Project limit: 3-63 letters, digits, dots, underscores or hyphens; starts/ends alphanumeric, no <code>..</code> , not an IPv4 address.

The multilingual E5 model runs on the CPU; query and document prefixes are handled by `embeddings.py`. The default model produces 384-dimensional vectors. A model change requires a compatible new index.

Compose overrides corpus/index paths with mounted container paths. A host `.env` path does not relocate those volumes. Increasing context length consumes more model/KV-cache memory and does not automatically improve relevance.

Authoritative definitions: `config.py`, `.env.example`, `embeddings.py`, `compose.yaml`. Print effective values with `uv run agentic-rag config`.

14 Configuration: workflow controls

Variable	Default	Meaning / constraint
TOP_K	4	Maximum relevant chunks retained per retrieval after grading; at least 1.
RETRIEVAL_CANDIDATES	20	Pool before grading; 1-100. Effective depth is at least TOP_K.
HYBRID_SEARCH	true	Fuse vector retrieval with local SQLite FTS5 BM25.
GRADE_WITH_LLM	true	One relevance-grading call per retrieval; skipped with fake LLM.
MAX_RETRIES	2	Maximum verification-driven re-plans; at least 0. Zero disables re-planning.
INGEST_ON_START	true	serve downloads missing sources and synchronizes the index before the UI.
LOG_LEVEL	INFO	DEBUG, INFO, WARNING or ERROR.

Controlled retrieval comparison

To approximate the previous vector-only retrieval configuration, use PowerShell:

```
$env:HYBRID_SEARCH = "false"
$env:RETRIEVAL_CANDIDATES = "4"
$env:TOP_K = "4"
uv run agentic-rag eval --dataset data/eval/questions.jsonl
```

For the current hybrid defaults, use `HYBRID_SEARCH=true` and `RETRIEVAL_CANDIDATES=20`, then re-run with the same corpus, chat model and judge. These settings reproduce a retrieval configuration, not all historical code behavior: verification and the tool fast path have also changed.

Library configuration discipline

Settings are frozen and suitable as cache keys. Derive a validated variant with

`Settings.model_validate(**settings.model_dump(), **changes)`; avoid `model_copy(update=...)`, which bypasses validation.

`get_settings()` caches one instance per process. Clear its cache when a test deliberately changes the environment. For a running UI, restarting is the reliable way to apply environment and corpus changes.

The lower-level `MIN_SCORES` thresholds and chunking defaults are code constants/configuration objects, not additional environment variables. Adding a new setting requires updating `Settings`, `.env.example`, tests and relevant container bindings together.

Authoritative definitions: `config.py`, `rag/graph.py`, `ingestion/chunking.py`.

15 UI, streaming and trace events

The Streamlit app keeps chat history in session state and caches the compiled graph by its settings. It submits completed history plus the new user message; interrupted turns are displayed but excluded from the next model conversation.

Streamed execution steps

The UI consumes synchronous graph updates with subgraph streaming enabled. Main steps appear as they finish; parallel workers are grouped by LangGraph step. Each search can show its RAG rewrite, candidate retrieval and retained evidence. Completed traces are collapsed by default; running traces remain visible.

This is workflow-step streaming. The current app delivers the final answer after graph completion; it does not stream answer tokens from an unverified draft. Source cards expose indexed context and an available commit revision.

Numbered answer citations become clickable HTTP(S) reference links. Citation formatting preserves code samples. The public URL and the indexed revision have different roles: one is a navigation link, the other identifies the evidence snapshot.

Shared instrumentation

`@traced` records a `TraceEvent` for each successfully completed node. Fields include `node`, `started_at`, `ended_at`, `duration_ms`, `summary`, `metadata` and optional `step`. Timing uses a monotonic high-resolution clock mapped to epoch time.

`run_rag_subtask` forwards subgraph events into the main trace. During live subgraph streaming, `trace_events_from_chunk(..., skip_forwarded=True)` prevents those events from appearing twice when the parent update arrives.

Do not sum parent RAG-worker duration and its child-node durations as independent service time. Parallel durations can overlap, so their sum is also different from end-to-end wall-clock latency.

Constraints when extending tracing

The decorator supports dictionary partial updates or `None`. Returning LangGraph `Command` requires explicit tracing support. A node-level `CachePolicy` would replay old trace timestamps; cache internal work instead so every execution emits a fresh event.

Implementation: `ui/app.py`, `ui/components.py`, `tracing.py`; no durable session store is configured.

16 Testing and continuous integration

Tests use fake providers, temporary data and controlled settings. The default pytest configuration excludes the live `ollama` marker. This provides an offline test suite without a GPU, live model or downloaded frontend corpus.

```
uv run ruff check .
uv run ruff format --check .
uv run pytest
```

Use a focused test file during development, then run the required full checks before a change is merged. Download tests create temporary local Git repositories and may be skipped if Git is unavailable.

Useful test areas

Test files	Behavior covered
<code>test_config.py</code> , <code>test_cli.py</code>	Defaults, validation, precedence, options and exit codes.
<code>test_ingestion.py</code> , <code>test_embeddings.py</code>	Cleaning, chunk boundaries, incremental index and model identity.
<code>test_rag_subgraph.py</code> , <code>test_rag_improvements.py</code>	Retrieval/grading, hybrid fusion, fallbacks and metadata migration.
<code>test_agent_graph.py</code> , <code>test_state.py</code>	Routing, fan-out, reducers, bounded re-plans and verification.
<code>test_tools.py</code> , <code>test_ui.py</code> , <code>test_tracing.py</code>	Deterministic results, UI rendering and trace forwarding.
<code>test_evaluation.py</code> , <code>test_loadtest.py</code>	Strict datasets, metrics, aggregation and reports.

Live model check

```
ollama pull qwen3.5:4b
uv run pytest -m ollama
```

The live test follows host Ollama settings and may skip when the server is unreachable. It is a connection/model integration check; functional answer quality is measured separately by `eval`.

CI runs on every pull request and push to `main`. It installs `uv` 0.12.6, synchronizes the lock, checks lint/format and runs offline tests. A separate job builds the Docker image and invokes the image's `agentic-rag --version`. No historical test count is treated as a current guarantee.

Basis: `pyproject.toml`, `tests/conftest.py`, `.github/workflows/ci.yml`.

17 Extending the application

Preserve explicit dependencies, typed inputs and evidence contracts when adding behavior. Change the smallest relevant module first and verify the route that reaches it.

Add or update a documentation source

1. Add a pinned `[[sources]]` entry to `data/sources.toml`, including license, file patterns and public URL mapping.
2. Check whether the existing Markdown cleaners support its dialect; add focused loader coverage if required.
3. Run `ingest --download`, inspect loaded metadata and retrieve representative questions.
4. Restart the graph/UI so the lexical snapshot is refreshed; add representative development evaluation questions.

Add a deterministic tool

Implement the domain operation independently of the model. Wrap it in a `BaseTool` with a strict Pydantic input schema and user-safe `ToolException` errors. Register it through `get_non_retrieval_tools(settings)` so it is available to routing and planning.

Review prompts and fake-provider behavior for the new route, then test correct invocation, invalid arguments and execution failures. The exact-tool fast path has an explicit three-tool allowlist: a new tool does not automatically bypass verification. Extend that allowlist only if repeating its successful output directly is appropriate and tested.

Add a graph node or state field

Update the state contract, implement a synchronous node with keyword-only dependencies, apply tracing, wire the graph and export its Mermaid diagram. For new `Send` workers, pass the appropriate input schema explicitly. Parallel writes need a deliberate reducer or distinct state fields.

If the change returns `Command`, adds checkpointing, changes citation numbering or introduces node caching, first revisit the tracing and state-reset assumptions documented in chapters 08 and 15.

Change models or dependencies

Keep the same judge and dataset for chat-model comparisons. An embedding-model change requires a new/rebuilt index and re-evaluation of score thresholds. Dependency updates must keep `pyproject.toml` and `uv.lock` consistent and pass the image build as well as local checks.

Extension points: `agent/tools.py`, `agent/prompts.py`, `llm.py`, `agent/graph.py`, `config.py`, `data/sources.toml`.

18 Functional evaluation

The strict UTF-8 JSONL loader validates one question per line. Required fields are `id`, `question` and `reference_answer`. Optional fields include expected intent/documents, evidence groups, fixed conversation history, tags and notes. Duplicate IDs, unknown fields, repeated keys and malformed values fail with a file/line error.

The development regression set has 17 questions. A separate 24-question holdout adds follow-ups, Hungarian queries, ambiguous APIs, version boundaries, unsupported APIs and mixed tool/retrieval requests. Keep it out of prompt tuning; its draft reference answers should receive domain-expert review before being treated as a quality benchmark.

```
uv run agentic-rag eval --dataset data/eval/questions.jsonl --judge-model qwen2.5:7b-instruct
uv run agentic-rag eval --dataset data/eval/holdout.jsonl --judge-model qwen2.5:7b-instruct
uv run agentic-rag eval --target node --node analyze_request
uv run agentic-rag eval --target node --node run_rag_subtask
```

Pull the chosen judge model before a live run. The judge option applies to full-graph evaluation. Isolated retrieval cannot resolve conversational history and rejects such dataset cases; use the full graph for follow-ups.

Metric	Interpretation
Routing accuracy	Chosen intent agrees with an expected intent.
Retrieval hit@k	At least one retrieval task has an expected document within its first k retained chunks.
Complete evidence@k	Every required evidence group is represented; alternatives inside one group are interchangeable.
Answer correctness	Judge comparison to the reference: 1, 0.5 or 0.
Faithfulness	Judge support check against retrieved/tool evidence: 1, 0.5 or 0.

Retrieval metrics use ranked post-grading task sources, not the de-duplicated answer citation list. Check applicability counts and individual errors alongside averages. Fake mode omits judged correctness/faithfulness and cannot establish real answer quality.

Reports are timestamped JSON plus Markdown in `data/eval/results/`, or `--output-dir`. They capture settings and per-item results. Keep the judge, corpus and model configuration fixed for meaningful comparisons.

Measured results

Chapter 19 compares the current defaults, measured on 4 October 2026, with the baseline of 3 October, and summarises the holdout. Every figure comes from a report in `data/eval/results/`.

```
Reports: development set eval-graph-20261004T191858Z, ...192448Z and ...193039Z (identical scores); holdout
eval-graph-20261004T194127Z; load loadtest-ollama-c4-20261004T195038Z and
loadtest-ollama-c1-20261004T195534Z.
```

19 Performance and measured results

The RAG reliability update was measured on 4 October 2026 against the baseline of 3 October, on the same machine (Windows 11, RTX 5070 Laptop GPU with 8 GB, Ollama 0.35.0 with its defaults, CPU E5 embeddings) and with the same model builds: `qwen3.5:4b 2a654d98e6fb` with thinking off, judged by `hf.co/bartowski/Qwen2.5-7B-Instruct-GGUF:Q4_K_M`. Ollama tags are mutable; check `ollama list` before comparing results.

Functional quality

Measure	Baseline	Update
Routing / hit@4 (17 questions)	1.00 / 0.90	1.00 / 1.00
Correctness / faithfulness	0.91 / 0.94	0.91 / 1.00
Holdout, 24 questions: correctness / faithfulness	—	0.67 / 0.73

Correctness stays at 0.91 because the judge misreads two correct answers (0.97 without them); on the holdout three judge errors hide 0.12 (0.79). The holdout fails on version boundaries, an ambiguous framework question and the answer language. With `HYBRID_SEARCH=false` and `RETRIEVAL_CANDIDATES=4` retrieval falls back to 0.90.

Load

Load run (one slot)	Throughput	p50 / p95
Baseline, 1 user, 50 requests	11.5/min	3.6 s / 14.9 s
Update, 1 user, 50 requests	11.7/min	3.9 s / 12.9 s
Baseline, 4 users, 100 requests	12.2/min	17.6 s / 35.4 s
Update, 4 users, 100 requests	12.0/min	20.4 s / 39.7 s

The bottleneck is still queued local LLM decoding. Exact tool answers cut a request to about 4.7 LLM calls (5.3 before), but grading 20 candidates costs 0.22 s per search, which shows under load. Parallel Ollama slots helped the tested 7B transformer, not this Qwen3.5 build.

```
uv run agentic-rag loadtest --requests 100 --concurrency 4
uv run agentic-rag loadtest --requests 50 --concurrency 1
```

Evidence: `docs/evaluation.md`, `docs/performance.md`, `docs/rag-improvements.md`, `data/eval/results/`.

20 Troubleshooting and runbook

Symptom	Action
Missing knowledge-base index	Run <code>ingest --download</code> ; check <code>DATA_DIR</code> and <code>CHROMA_DIR</code> with <code>config</code> . Preparation errors can leave the UI running with no usable index.
Embedding mismatch	Select the index's original provider/model or run <code>ingest --rebuild</code> . Startup preparation also rebuilds a mismatched index.
Ollama connection refused	Check that the server is running and <code>OLLAMA_BASE_URL</code> points to the host or Compose service appropriate to the process.
Model not found	Pull <code>OLLAMA_MODEL</code> on the server used by the app; a host pull does not populate a separate container volume.
Browser-support tool fails	Download the corpus and confirm <code>DATA_DIR/browser-compat-data</code> exists. Contrast/specificity do not need that dataset.
Old sources after ingestion	Restart the app or create a new graph to refresh cached resources and the lexical snapshot.
Invalid or unreadable <code>.env</code>	Save as UTF-8, inspect named validation errors and remove conflicting shell overrides.
Slow first request / startup	Inspect download/loading logs and caches; distinguish first-use initialization from warm inference.
Poor framework-specific answers	Inspect rewrite, retained chunks and expected evidence; confirm source selection and version before tuning prompts.
Unverified draft withheld	Retry the request and inspect verifier parsing logs. <code>unavailable</code> deliberately withholds the draft.

Operational commands

```
uv run agentic-rag config
docker compose ps
docker compose logs --tail 100 app
docker compose logs --tail 100 ollama
docker compose run --rm --no-deps app agentic-rag ingest --rebuild
```

CLI conventions: 0 means successful command completion; 1 covers anticipated operational failures such as a missing dataset/corpus or a download failure; 2 covers usage/configuration errors; 130 means interruption. A successful evaluation command can still contain failed question results, so inspect the report's error fields.

Unexpected programming exceptions retain tracebacks. Avoid broad exception handling that hides them as a user mistake. Do not delete persistent volumes as the first diagnostic step; use targeted ingestion or model/configuration repair.

21 Maintenance and source reference

This guide describes project version 0.1.0 as of 4 October 2026, including the RAG reliability update. The implementation is the authority where an older planning document or an earlier benchmark differs from current behavior.

Reference	Consult for
<code>README.md</code> , <code>README.hu.md</code>	Project motivation, setup, design decisions and measured results.
<code>docs/architecture.md</code>	Detailed workflow, state and dependency contracts.
<code>docs/rag-improvements.md</code>	The update: hybrid retrieval, verification, holdout, migration and measured results.
<code>docs/evaluation.md</code>	Functional measurements (baseline, update, holdout), judge reliability and reproducibility.
<code>docs/performance.md</code>	Load reports (baseline and update), per-node times and bottleneck analysis.
<code>data/README.md</code> , <code>data/eval/README.md</code>	Corpus layout, strict dataset schema and report interpretation.
<code>config.py</code> , <code>.env.example</code>	Authoritative runtime settings and validation rules.
<code>agent/</code> , <code>rag/</code> , <code>ingestion/</code>	Authoritative runtime and ingestion behavior.
<code>compose.yaml</code> , <code>Dockerfile</code> , <code>.github/workflows/ci.yml</code>	Container behavior, pinned build tooling and CI gates.

Keep the language editions aligned

Update both Markdown sources when contracts or commands change. Keep identifiers and commands unchanged across translations; translate explanatory prose. Recheck configuration against `Settings`, inspect CLI `--help`, validate dataset counts, then render and review both PDFs.

The sources are `docs/developer-guide.en.md` and `docs/developer-guide.hu.md`. The builder is `docs/build_developer_pdfs.py`; it writes `output/pdf/developer-guide-en.pdf` and `output/pdf/developer-guide-hu.pdf`.

The document builder needs ReportLab and either Arial, Georgia and Consolas (Windows) or DejaVu Sans, Serif and Sans Mono; its layout follows the UI theme in `.streamlit/config.toml`. These are document-tooling dependencies, separate from the chatbot's runtime dependencies. Run `python docs/build_developer_pdfs.py` from the project root after installing the tooling in a suitable environment.

Review pagination, table wrapping, command legibility and Hungarian accents after regeneration. New results should identify the dataset, corpus snapshot, chat/judge models and serving settings; do not silently replace historical results with estimates.

Document validation checks the guide and its examples; it does not replace a new live answer-quality evaluation or performance benchmark.